

Quiz 1 (model answers)

In each question, tick **all** the answers that are correct.

If no answer is correct, leave all boxes unticked.

Each question is worth 1 point; the one with a * is a bonus.

Your name: _____

1. What is the name of the course?

- ☐ Computer Programming 2
- ☐ The Art of Computer Programming 2
- ☐ The Craft of Computer Programming 3
- ☐ Philosophy of Computation

2. Who is teaching this course?

- ☒ Anton Gnatenko
- ☒ Pierre Senellart
- ☐ Pierr Senellar
- ☐ Antoine Garnier
- ☐ Piotr Senelarko

3. Which of the following functions are computable?

- ☒ $f_1: \mathbb{N} \rightarrow \mathbb{N}$, where $f_1(n) = n^n$;
- ☒ $f_2: \mathbb{N} \rightarrow \mathbb{N}$, where $f_2(n) = n - 1$, for $n > 0$, and $f_2(0)$ is undefined;
- ☒ $f_3: \{a, b, c, d\}^* \rightarrow \{a, b, c, d\}^*$, where $f_3(w)$ is the reverse of w (e.g. $f_3(abcb) = bcba$);
- ☒ f_4 , which takes a graph as input and outputs 1 if this graph is a tree and 0 otherwise;
- ☐ f_5 , which takes a Python program and an input file, and outputs 1 if the program terminates on that file, and 0 if it runs forever;

4*. Suppose we are given a magic program that computes the halting function. We can use this program as a subprogram in other programs. Will there still be any uncomputable functions?

- ☒ Yes, some functions will still be uncomputable
- ☐ No, everything is computable now
- ☐ We can't tell for sure

How hard is this quiz, from 0 (very easy) to 10 (very hard)?

1 Explanations

Question 3. The function f_5 is *not* computable: it is the halting function. The functions f_1, f_2, f_3, f_4 of Question 3 are all computable, as explained below.

f_1 (computes n^n):

```
def f1(n):
    return n ** n
```

f_2 (computes $n - 1$, undefined at 0):

```
def f2(n):
    if n == 0:
        while True:
            pass          # loop forever: f_2(0) is undefined
    return n - 1
```

f_3 (computes the reverse of a word over $\{a, b, c, d\}$): build a new string by prepending each symbol of the input:

```
def f3(w):
    rev = ""
    for c in w:
        rev = c + rev    # put each symbol in front
    return rev
```

f_4 (says whether a graph on vertices $0, \dots, n - 1$ is a tree).

Recall that a *tree* is a connected graph with no cycles. A program can check connectedness, for instance by depth-first search, and the absence of cycles, for instance by detecting when a vertex is revisited during the search.

Question 4* (Bonus). Suppose, a program `magic_halt` computes the halting function

$$h(\pi, w) = \begin{cases} 1, & \text{if } \pi \text{ halts on input } w, \\ 0, & \text{if } \pi \text{ loops forever on input } w. \end{cases}$$

Consider a function h' that is like the halting function but returns 2 and 3 instead of 1 and 0:

$$h'(\pi, w) = \begin{cases} 2, & \text{if } \pi \text{ halts on input } w, \\ 3, & \text{if } \pi \text{ loops forever on input } w. \end{cases}$$

Without the magic program, h' is uncomputable as h itself. But with the magic program, we can compute h' :

```
def h_prime( $\pi$ , w):  
    if h( $\pi$ , w) == 1:  
        return 2  
    else:  
        return 3
```

In general, however the cardinality argument of Lecture 1 still applies. There are only countably many programs, but are uncountably many functions.